

LABORATORY 9

Name: Mar Julius Ubas

Course: BSIT-2C

Lab-9

I. Objectives

Implement Spring Security for REST API protection.

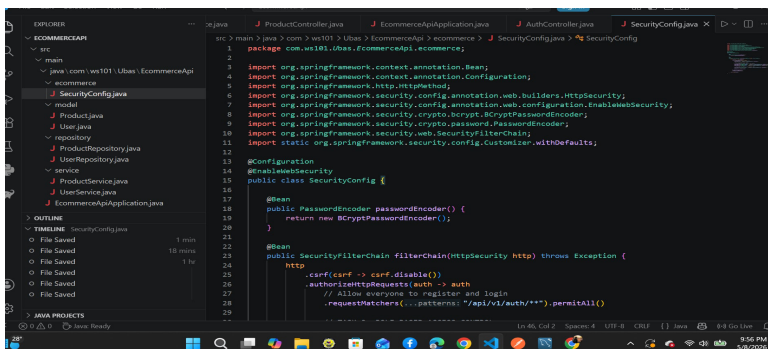
Use BCrypt for password hashing.

Verify user authentication and authorization via Postman.

II. Documentation of Results

1. Security Configuration (VS Code)

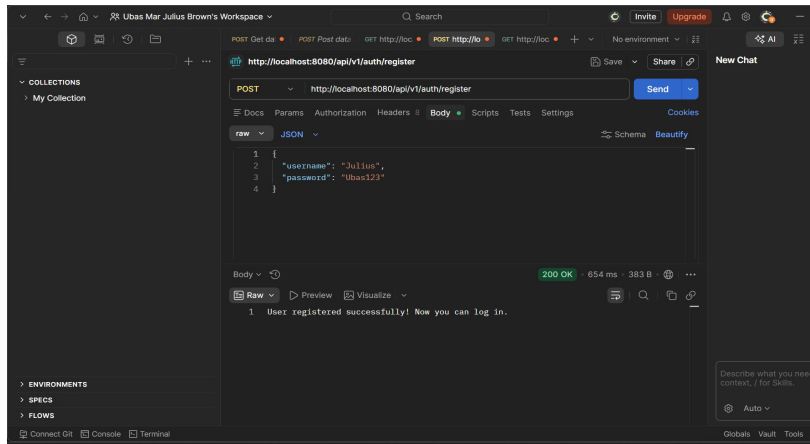
Description: This screenshot shows the SecurityConfig.java file where the Security Filter Chain is defined. It shows which endpoints are public (/register) and which require authentication.



```
1 package com.w181.ubas.EcommerceApi;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.http.HttpMethod;
6 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
8 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
9 import org.springframework.security.crypto.password.PasswordEncoder;
10 import org.springframework.security.web.SecurityFilterChain;
11 import static org.springframework.security.config.Customizer.withDefaults;
12
13 @Configuration
14 @EnableWebSecurity
15 public class SecurityConfig {
16
17     @Bean
18     public PasswordEncoder passwordEncoder() {
19         return new BCryptPasswordEncoder();
20     }
21
22     @Bean
23     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
24         http
25             .csrf(conf -> conf.disable())
26             .authorizeHttpRequests(auth -> auth
27                 // Allow everyone to register and login
28                 .requestMatchers(...pattern: "/api/v1/auth/**").permitAll()
29             )
30     }
31 }
```

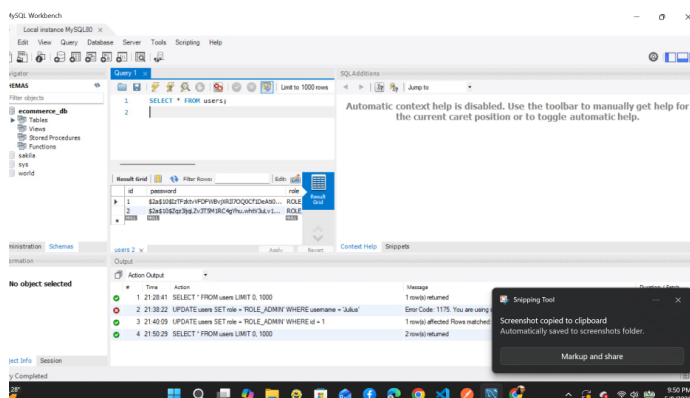
2. User Registration (Postman POST)

Description: Performing an HTTP POST request to /api/v1/auth/register. This proves the backend can receive user data and process it through the password encoder.



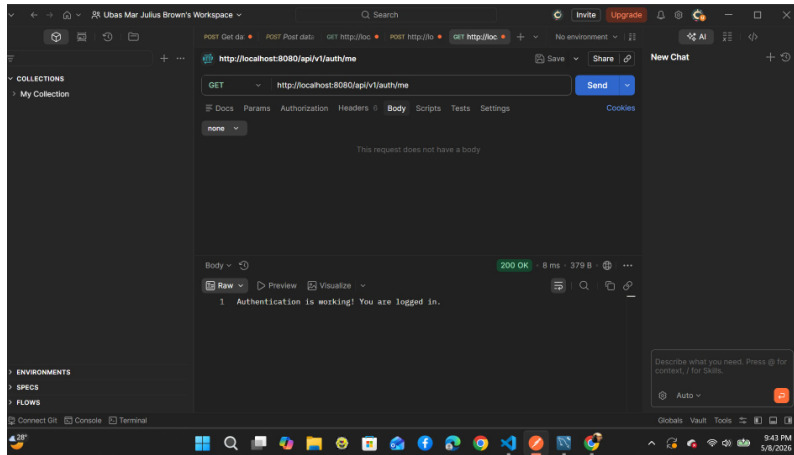
3. Encrypted Data Storage (MySQL Workbench)

Description: This shows the users table in the database. The password column proves that Bcrypt is working because the passwords are saved as secure hashes rather than plain text.



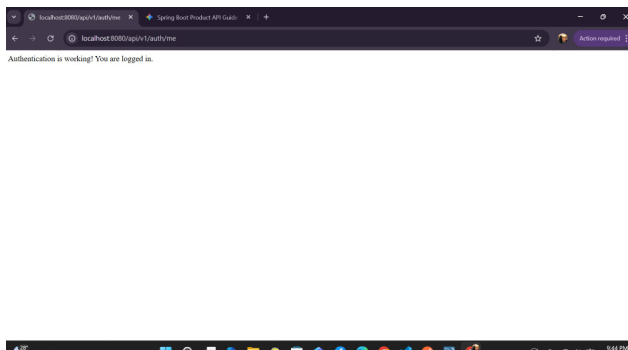
4. Authentication Verification (Postman GET)

Description: A GET request to `/api/v1/auth/me` using Basic Auth. The 200 OK status and response message confirm that the system successfully identifies the logged-in user.



5. Browser Security Check (Chrome)

Description: Testing the protected endpoints in the browser. This confirms that the security filters correctly trigger the login prompt or block unauthorized access at the browser level.



III. Conclusion

The implementation of Spring Security successfully secured the Ecommerce API. By using BCrypt, sensitive user data is protected in the database. The testing across Postman and the Browser confirms that the authentication middleware is correctly intercepting requests and validating user credentials before allowing access to protected resources.

